Meteora v2 - 0.1.5

Security Assessment

October 15th, 2025 — Prepared by OtterSec

Gabriel Ottoboni ottoboni@osec.io

Akash Gurugunti sud0u53r.ak@osec.io

Robert Chen r@osec.io

Table of Contents

01 — Executive Summary

Overview

Meteora engaged OtterSec to assess the `damm-v2` program. This assessment was conducted between August 21st and August 29th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 2 findings throughout this audit engagement.

In particular, we identified a vulnerability where the swap validation check, which enforces a single swap instruction per transaction, may be bypassed, as it only verifies the swap discriminator and ignores swap2 calls, allowing multiple swaps to execute within a single transaction (OS-MCP-ADV-00). Additionally, the swap instructions incorrectly utilize the raw input amount instead of the post-transfer-fee amount, resulting in inaccurate accounting (OS-MCP-ADV-01).

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/MeteoraAg/cp-amm. This audit was performed against PR#94.

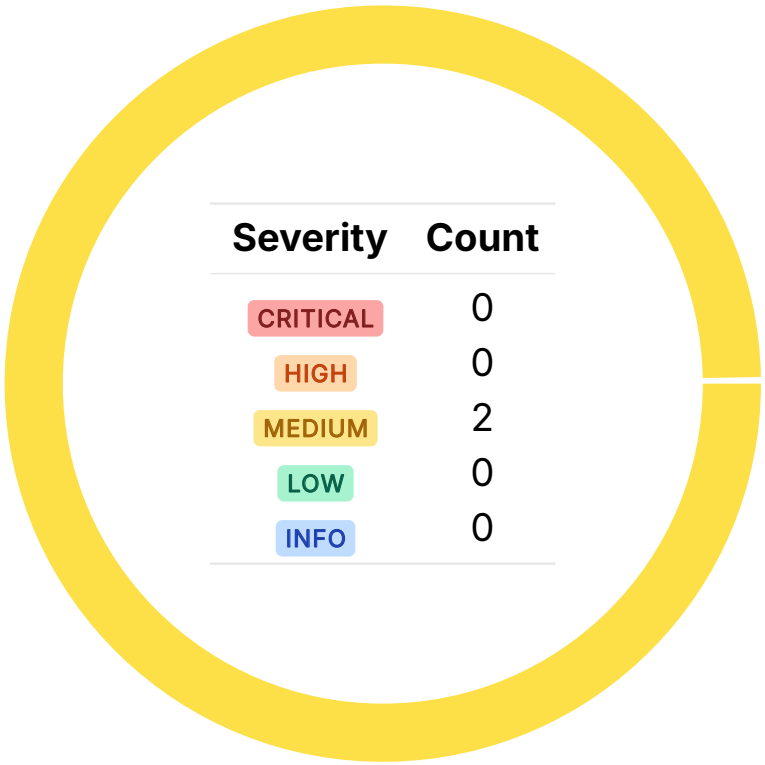
A brief description of the program is as follows:

Name	Description
damm-v2	an upgraded automated market maker that enhances dynamic-amm v1 with unique pool accounts, Token2022 support, and flexible fee mechanisms.

03 — Findings

Overall, we reported 2 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	2
LOW	0
INFO	0

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-MCP-ADV-00	MEDIUM	RESOLVED	The swap validation check to limit a single swap instruction per transaction only checks for the `swap` discriminator, ignoring `swap2` calls, allowing the execution of multiple swaps in a single transaction.
OS-MCP-ADV-01	MEDIUM	RESOLVED	`swap_exact_in` and `swap_partial_fill` incorrectly utilize the raw input amount instead of the post-transfer-fee amount, resulting in inaccurate accounting.

Single Swap Instruction Check Bypass MEDIUM OS-MCP-ADV-00

Description

The validator only compares instruction data against a single swap discriminator (`SwapInstruction::DISCRIMINATOR`), ignoring the `swap2` instruction discriminator. Consequently, the function fails to recognize a `swap2` call as a swap. As a result, multiple swaps may be executed on the same pool within a single transaction, one through `swap` and one (or more) through `swap2`, bypassing the core purpose of `validate_single_swap_instruction`. This issue also seems to be present in DBC.

>_ cp-amm/src/instructions/swap/ix_swap.rs RUST

```rust
pub fn validate_single_swap_instruction<'c, 'info>(
    pool: &Pubkey,
    remaining_accounts: &'c [AccountInfo<'info>],
) -> Result<()> {
    [...]
    if current_instruction.program_id != crate::ID {
        // check if current instruction is CPI
        // disable any stack height greater than 2
        if get_stack_height() > 2 {
            return Err(PoolError::FailToValidateSingleSwapInstruction.into());
        }
        // check for any sibling instruction
        let mut sibling_index = 0;
        while let Some(sibling_instruction) = get_processed_sibling_instruction(sibling_index) {
            if sibling_instruction.program_id == crate::ID
                && sibling_instruction.data[..8].eq(SwapInstruction::DISCRIMINATOR)
            {
                if sibling_instruction.accounts[1].pubkey.eq(pool) {
                    return Err(PoolError::FailToValidateSingleSwapInstruction.into());
                }
            }
            sibling_index = sibling_index.safe_add(1)?;
        }
    }
    [...]
}
```

Remediation

Ensure to check for all swap-related discriminators.

Patch

Resolved in 773ae78.

Incorrect Input Fee Handling MEDIUM OS-MCP-ADV-01

Description

In `swap_exact_in` and `swap_partial_fill`, the pool pricing functions (`get_swap_result_from_exact_input` / `get_swap_result_from_partial_input`) utilize the raw user-supplied `amount_in` instead of the effective `excluded_transfer_fee_amount_in` that actually arrives in the pool. Consequently, the pool overestimates input tokens, incorrectly calculating the output amounts, resulting in potential slippage violations and accounting inconsistencies.

>_ cp-amm/src/instructions/swap/swap_exact_in.rs RUST

```rust
pub fn process_swap_exact_in<'a, 'b, 'info>(
    params: ProcessSwapParams<'a, 'b, 'info>,
) -> Result<ProcessSwapResult> {
    [...]
    let swap_result =
        pool.get_swap_result_from_exact_input(amount_in, fee_mode, trade_direction,
            ↪  current_point)?;
    [...]
}
```

>_ cp-amm/src/instructions/swap/swap_partial_fill.rs RUST

```rust
pub fn process_swap_partial_fill<'a, 'b, 'info>(
    params: ProcessSwapParams<'a, 'b, 'info>,
) -> Result<ProcessSwapResult> {
    [...]
    let swap_result = pool.get_swap_result_from_partial_input(
        amount_in,
        fee_mode,
        trade_direction,
        current_point,
    )?;
    [...]
}
```

Remediation

Utilize `excluded_transfer_fee_amount_in` instead of `amount_in` in `swap_exact_in` and `swap_partial_fill`.

Patch

Resolved in fc12297.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.